四、SQL计算

设某单位有一个E-P数据库，包括员工表Employees、项目表 Projects、参与项目表EP 3个关系模式:

Employees(ENO,ENAME,ESex, EAge, EDept,ELevel);

Projects(PN0,PNAME, Duration, Budgets, StartDate, EndDate);

EP(ENO,PNO,Roles,Scores):

其中，

员工表 Employees 由员工编号(EN0)，员工姓名(ENAME)，员工性别(ESex)，员工年龄(EAge)，员工部门(EDept)，员工级别(ELevel)组成，主键是EN0;

项目表Projects 由项目编号(PNO)，项目名称(PNAME)，工期(Duration，数值型:单位人月)，项目预算(Budgets，数值型)，开始日期(StartDate,日期型)，结束日期(EndDate,日期型)组成，主键是 PNO;

参与项目表 EP由员工编号(ENO),项目编号(PNO),担任角色(Roles)，完成业绩(Scores,数值型:0-100)组成，主键是ENO+PNO:

3 个关系模式满足语义:一个员工可以参与多个项目，一个项目可以有多个员工参加。使用 SQL 完成以下各操作:

假设有以下关系模式：

- **员工表 (Employees)**：`ENO, ENAME, ESex, EAge, EDept, ELevel`
- **项目表 (Projects)**：`PNO, PNAME, Duration, Budgets, StartDate, EndDate`
- **参与项目表 (EP)**：`ENO, PNO, Roles, Scores`

(1)查询"研发"部门所有员工参与的，工期大于等于5人月的员工姓名，项目名称，以及担任角色。

```
SELECT E.ENAME, P.PNAME, EP.Roles
FROM Employees E
JOIN EP ON E.ENO = EP.ENO
JOIN Projects P ON EP.PNO = P.PNO
WHERE E.EDept = '研发' AND P.Duration >= 5;
```

(2)查询"测试"部门女员工参与的项目的最高业绩以及平均业绩。

```
SELECT MAX(EP.Scores) AS MaxScore, AVG(EP.Scores) AS AvgScore
FROM Employees E
JOIN EP ON E.ENO = EP.ENO
WHERE E.EDept = '测试' AND E.ESex = '女';
```

(3)查询没有参加项目"电子商务网站开发"的员工编号，姓名及级别。

```sql
SELECT E.ENO, E.ENAME, E.ELevel
FROM Employees E
WHERE E.ENO NOT IN (
    SELECT EP.ENO
    FROM EP
    JOIN Projects P ON EP.PNO = P.PNO
    WHERE P.PNAME = '电子商务网站开发'
);
```

(4)查询每个员工级别完成的项目的数量，人员个数，及最大项目业绩，按员工级别分组统计。

```sql
SELECT E.ELevel, COUNT(DISTINCT EP.PNO) AS ProjectCount, COUNT(DISTINCT EP.ENO) AS EmployeeCount,
MAX(EP.Scores) AS MaxScore
FROM Employees E
JOIN EP ON E.ENO = EP.ENO
GROUP BY E.ELevel;
```

(5)查询每个员工及其参与项目情况，包括没有参与任何项目的员工，按完成业绩降序排列。

```sql
SELECT E.ENO, E.ENAME, EP.PNO, EP.Scores
FROM Employees E
LEFT JOIN EP ON E.ENO = EP.ENO
ORDER BY EP.Scores DESC;
```

(6)项目“美食 APP”取消，删除所有该项目在 EP 表中的记录。

```sql
DELETE FROM EP
WHERE PNO = (SELECT PNO FROM Projects WHERE PNAME = '美食 APP');
```

(7)建立员工视图,满足条件为:员工级别是“数据库管理员”，参与开始日期在 2019.1.1号后的项目，并且完成业绩在 85 分以上，只列出员工编号，员工姓名，员工部门即可。

```sql
CREATE VIEW DatabaseAdmins AS
SELECT E.ENO, E.ENAME, E.EDept
FROM Employees E
JOIN EP ON E.ENO = EP.ENO
JOIN Projects P ON EP.PNO = P.PNO
WHERE E.ELevel = '数据库管理员' AND P.StartDate > '2019-01-01' AND EP.Scores > 85;
```

(8)查询参加了3号项目，并且业绩超过该项目平均业绩的员工编号，姓名，和业绩。

```sql
SELECT EP.ENO, E.ENAME, EP.Scores
FROM EP
JOIN Employees E ON EP.ENO = E.ENO
WHERE EP.PNO = 3 AND EP.Scores > (
    SELECT AVG(Scores)
    FROM EP
    WHERE PNO = 3
);
```

(9) 将员工级别是“数据库管理员”的所有员工的项目业绩提高20%。

```sql
UPDATE EP
SET Scores = Scores * 1.2
WHERE ENO IN (
    SELECT E.ENO
    FROM Employees E
    WHERE E.ELevel = '数据库管理员'
);
```

(10)查询参与全部项目的员工姓名与级别。

```sql
SELECT E.ENAME, E.ELevel
FROM Employees E
WHERE NOT EXISTS (
    SELECT P.PNO
    FROM Projects P
    WHERE NOT EXISTS (
        SELECT 1
        FROM EP
        WHERE EP.PNO = P.PNO AND EP.ENO = E.ENO
    )
);
```

五、简答题解答

1. 数据库系统具有数据与程序的独立性。什么是数据与程序的物理独立性?什么叫数据与程序的逻辑独立性?

- 数据与程序的物理独立性：
  - 物理独立性指的是数据库的物理存储方式的改变不会影响到应用程序的操作。即使数据的存储方式、存储结构发生改变，应用程序的逻辑层面不受影响，程序不需要进行修改。这种独立性保证了数据库能够适应硬件变化，增加了系统的灵活性和可扩展性。
- 数据与程序的逻辑独立性：
  - 逻辑独立性指的是数据库的逻辑结构的改变（如表结构、字段等）不应该影响到应用程序的操作。即使数据库中数据表的结构发生了修改，应用程序也无需进行调整。逻辑独立性保证了数据模型的变化不会影响到应用程序的业务逻辑和用户的使用体验。

2. 试述数据库安全控制的常用方法和技术。

数据库安全控制的常用方法和技术包括：

- **身份验证**：通过用户名和密码、指纹、智能卡等方式确认用户身份，确保只有授权用户才能访问数据库。
- **访问控制**：通过权限管理和角色控制，限定不同用户对数据库中不同数据的访问权限。常见的访问控制方式有基于角色的访问控制（RBAC）和基于属性的访问控制（ABAC）。
- **数据加密**：对数据库中的敏感数据进行加密，防止数据在传输和存储过程中的泄露。加密可以分为透明加密和应用级加密。
- **审计与监控**：通过日志记录和审计，跟踪所有对数据库的访问和操作，确保能够及时发现并响应任何异常活动或安全威胁。
- **备份与恢复**：通过定期备份数据库和实现灾难恢复机制，保证数据的安全性和可用性，防止数据丢失。

3. 试述事务的概念及事务的4个特性。恢复技术能保证事务的哪些特性?并发控制技术能保证事务的哪些特性?

- **事务的概念**：
  - 事务是数据库管理系统中一组操作的集合，这些操作要么全部执行成功，要么全部失败回滚。事务保证了数据库的完整性和一致性，并且与其他事务并行执行时不会相互干扰。
- **事务的四个特性（ACID特性）**：
  - **原子性（Atomicity）**：事务中的所有操作要么全部执行，要么全部不执行。即事务不可分割。
  - **一致性（Consistency）**：事务的执行必须使数据库从一个一致性状态变到另一个一致性状态，数据库约束必须保持不变。
  - **隔离性（Isolation）**：并发执行的事务应该相互隔离，一个事务的执行不应受到其他事务执行的影响。
  - **持久性（Durability）**：一旦事务提交，对数据库的修改是永久性的，即使系统崩溃也不会丢失。
- **恢复技术能保证事务的哪些特性**：
  - 恢复技术主要保证**原子性**和**持久性。**通过日志记录和事务回滚机制，恢复技术确保事务能够在发生系统崩溃或错误时正确恢复，保持事务的原子性（全有或全无）和持久性（提交后的数据不会丢失）。
- **并发控制技术能保证事务的哪些特性**：
  - 并发控制技术主要保证**隔离性。**通过锁机制、时间戳等方法，确保多个事务并发执行时，互不干扰，从而防止出现脏读、不可重复读、幻读等问题，保证事务的隔离性。

4. 登记日志文件时为什么必须先写日志文件，后写数据库?

- 解释：
  - 必须先写日志文件再写数据库，主要是为了保证**事务的原子性和持久性。**事务提交时，数据库中的更改应该是不可回滚的（持久性），但在提交之前，必须确保日志已被成功写入。如果事务提交后数据库更新发生故障，日志文件中的记录可以用于恢复未提交的事务数据，并回滚已提交的事务，从而保证数据库的完整性和一致性。先写日志文件，确保事务操作可以追溯，避免数据丢失。

5. 多粒度封锁方法中，为什么要引进意向锁?意向锁的含义是什么?

- **引进意向锁的原因**：
  - 在多粒度封锁方法中，意向锁（Intention Lock）用于**减少锁冲突**，特别是在锁定数据库中的大型资源（如表）时。意向锁的引入避免了对每个数据行、页或块进行单独加锁所带来的性能开销。它允许系统在较高层次（如表级别）了解锁的意图，从而避免不必要的阻塞和死锁。
- **意向锁的含义**：

- 意向锁是一种用于表示事务在更细粒度（如行或页）上请求锁定的意图的锁。在表级别加一个意向锁（如意向共享锁 IS 或意向排他锁 IX），表明事务将来可能会对该表中的某些行加共享锁或排他锁。通过意向锁，数据库可以协调多个事务之间的并发访问，避免全表级别的锁冲突。